Time series

Chicago transit ridership

LECTURE 15 | GÉRON CH. 13

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence · Third year

A new kind of data

Everything so far assumed the rows were exchangeable: a random split was valid because any row was as good as any other for training.

> **NOT HERE**
>
> In a time series the rows are **ordered**, and neighbouring rows are alike: one day's ridership tells you most of the next day's. Split such data at random and a point's own near-future lands in the training set — so the score you report is not the score you would get in deployment, and it is optimistic by a margin we will measure.

So the split changes, the baselines change, and the first thing to derive is what makes a series predictable at all.

Today

1. The brief, the series, and what a plot of it already tells you (15 min)
2. **The mathematics** — stationarity, differencing and autocorrelation (20 min)
3. **The method** — naive baselines, a linear model on lags, and backtesting on a rolling origin (40 min)
4. ARMA and its relatives, for context (10 min)

PART ONE

The brief

8 minutes

You have just been hired

You are a data scientist at the transit authority of a large city.

> **THE TASK**
>
> Forecast the number of people who will board **rail** and **bus** on the following day. You have daily boarding totals going back twenty years.

One number per mode, per day, one day ahead.

The first question is never technical

What decision does this number change?

- How many train sets leave the depot
- How many drivers are rostered, and at what notice
- Whether a spare crew is held at each terminus

Those decisions are made the evening before. That fixes the horizon at **one day**, and it fixes when the forecast has to exist: *before* the day it describes.

The two errors are not equal

Forecast too high

Empty vehicles run. The cost is fuel, crew hours and a line in the budget. It is measurable and it is survivable.

Forecast too low

Passengers are left on platforms at peak. The cost is political, and it does not appear in any table you will be shown.

***✗* We are about to choose a symmetric metric anyway, and we should say so out loud rather than discover it later.**

Frame the problem

Question	Answer	Because
Supervision	✓ **supervised**	the day arrives, and then we know
Task	✓ **regression**	a count, not a category
Learning mode	✓ **batch, offline**	one row per day; the whole record fits in memory

> ***THE LABEL IS FREE***
>
> *Nobody has to annotate anything. The next day's value is the label for today's window. Chapter 13 calls this the trick that makes a series into a supervised dataset.*

What is different from every application so far

The rows are **not exchangeable**.

- Row 400 is not a fresh draw from the same distribution as row 10. It is the same city, eighteen months later
- Row 400 and row 401 are almost the same measurement made twice
- The thing we are asked to predict is, by construction, the row we do *not* have

Hold on to that. It has consequences we will not reach today.

PART ONE, CONTINUED

The data

7 minutes

Fetch it with a function, not a download

```
from pathlib import Path
import tarfile, urllib.request
import pandas as pd

def load_ridership():
    tarball = Path("datasets/ridership.tgz")
    if not tarball.is_file():
        Path("datasets").mkdir(parents=True, exist_ok=True)
        url = "https://github.com/ageron/data/raw/main/ridership.tgz"
        urllib.request.urlretrieve(url, tarball)
        with tarfile.open(tarball) as t:
            t.extractall(path="datasets", filter="data")
    return pd.read_csv(
        "datasets/ridership/CTA_-_Ridership_-_Daily_Boarding_Totals.csv",
        parse_dates=["service_date"])
```

Same shape as the loader in the first application. The data will change; you will be on another machine; write the function.

Four lines of tidying, and one of them matters

```
df = load_ridership()
df.columns = ["date", "day_type", "bus", "rail", "total"]
df = df.sort_values("date").set_index("date")
df = df.drop("total", axis=1)      # it is just bus + rail
df = df.drop_duplicates()          # and this one removes 62 rows
```

The file ships with **7,701** rows and **7,639** distinct ones. Sixty-two days appear twice, identically. Nothing warns you.

✓ Reviewer question 4, from the first application: *what was dropped — rows, columns, NaNs? Count them.*

Here the answer is 62, and you only know because you counted.

What one row looks like

```
>>> df.head(3)
           day_type     bus    rail
date
2001-01-01        U  297192  126455
2001-01-02        W  780827  501952
2001-01-03        W  824923  536432
```

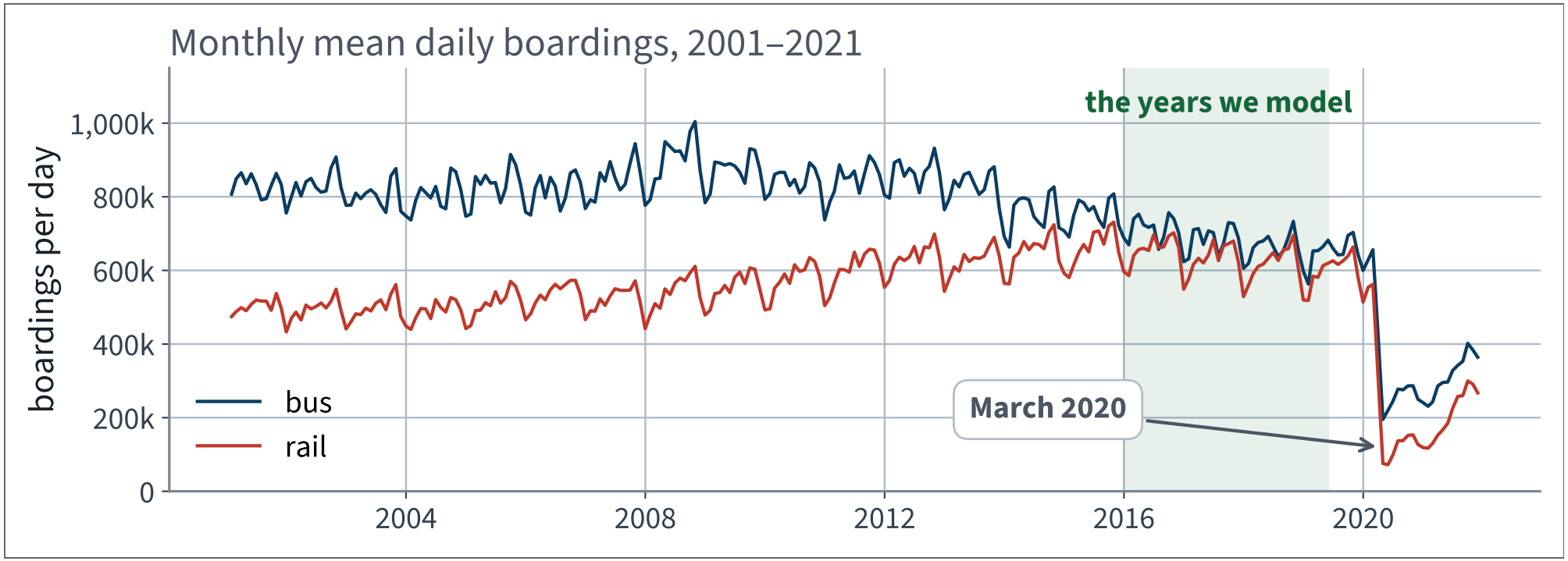
On the first day of 2001, **297,192** people boarded a bus and **126,455** boarded a train. It was a public holiday, which is why both numbers are low — and which the `day_type` column is telling us.

The one categorical column

day_type	Meaning	Days
W	an ordinary working day	5,336
U	the second weekend day, and public holidays	1,216
A	the first weekend day	1,087

This column is unusual and worth pausing on: we know it **in advance**. The next day's type is on the calendar today, so it is a legitimate input to a forecast of that day. Most features are not like that.

Plot it before you do anything else



Three things are visible at this scale and none of them is the thing we will model.

What that plot says

1. **Bus is falling, rail rose and then fell.** The two modes are not the same problem
2. **There is an annual wobble** on both, of the order of a tenth of the level
3. **March 2020 is a cliff**, not a dip

   A model fitted across that cliff is being asked to describe two different cities. We will come back to it, but not by pretending it is noise.

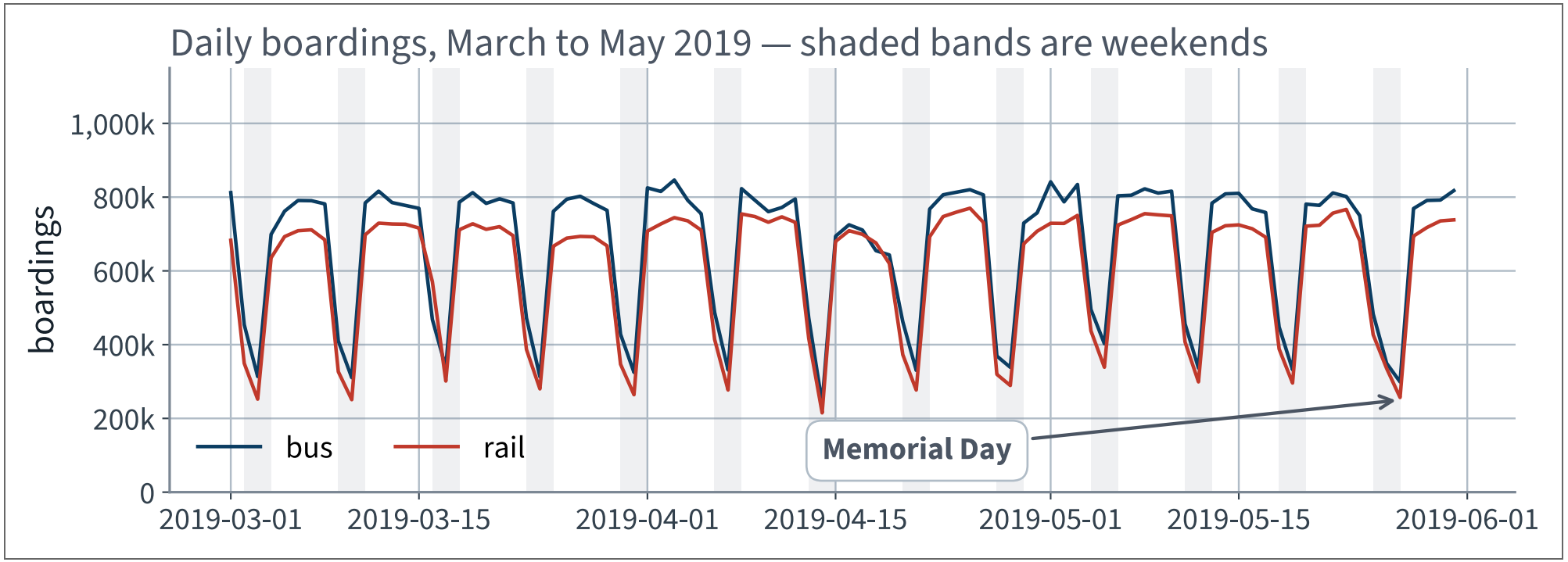
The decision that follows

> **MODEL 2016 TO THE MIDDLE OF 2019**
>
> Four years of one regime: 1,247 days, ending 31 May 2019. The book uses the same window, and for the same reason.

Everything after that date is set aside. Not because it is uninteresting — it is the most interesting part of the record — but because we cannot both use it and claim to have been surprised by it.

Data snooping, from the first application, applies to time as well as to rows.

Now zoom in until you can see individual days



A seven-day cycle so strong that it dominates everything else on the plot.

Observation 1 · A seven-day cycle

- Working days sit near the top of the range
- The two weekend days drop to roughly half
- The pattern repeats **every seven days**, almost exactly

> A repeating pattern with a fixed period is called **seasonality**. The period here is a week, not a year — the word has nothing to do with the seasons.

✓ This immediately suggests a forecast that requires no model at all. Hold that thought for ten minutes.

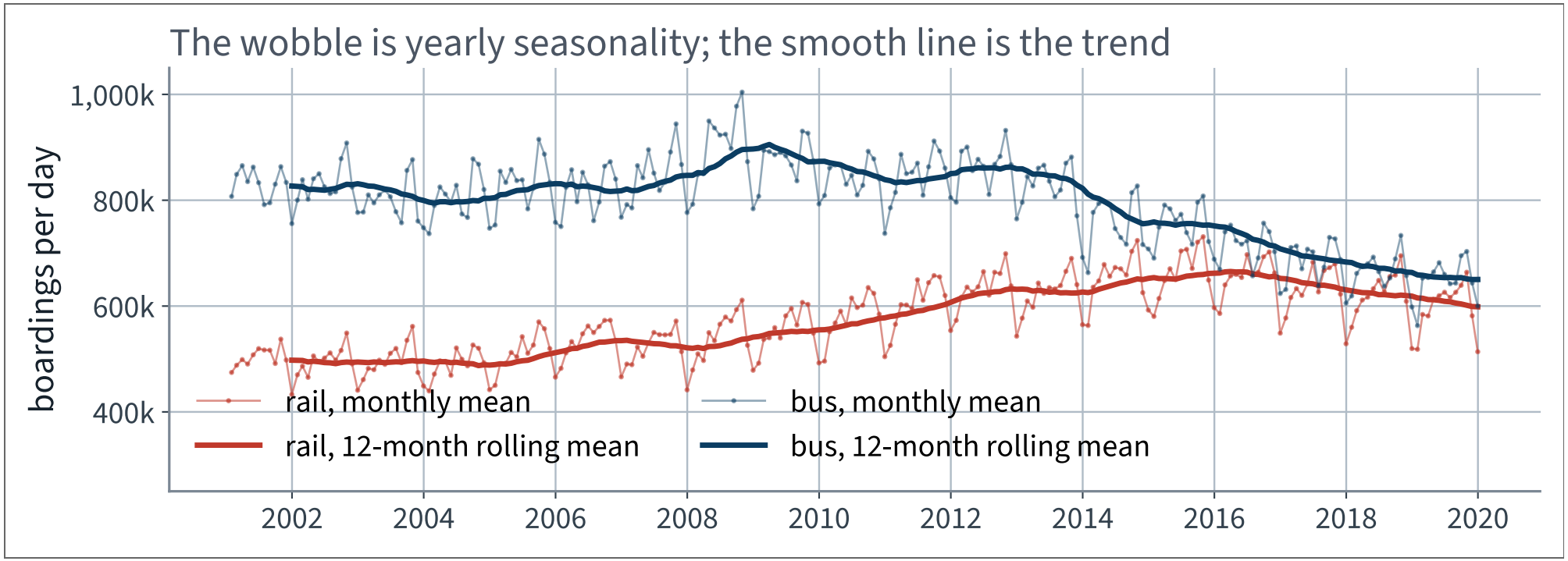
Observation 2 · One trough is in the wrong place

At the end of May there is a deep trough in the middle of a working week. Ask the data rather than the calendar:

```
>>> list(df.loc["2019-05-25":"2019-05-27"]["day_type"])
['A', 'U', 'U']
```

Three consecutive non-working days: a public holiday extending the weekend. The column we were given already knows.
Every anomaly you find should end either in an explanation or in a note that you could not find one. Not in silence.

And a second, slower cycle



The thin line wobbles once a year; the thick line is where the level is actually going.

Observation 3 · Seasonality and trend are different things

Seasonality

A pattern that repeats with a fixed period. Here: seven days, strongly; twelve months, weakly.

Trend

A slow movement of the level itself, with no period. Here: rail rising to about 2015, then falling.

A moving average of the right width removes the seasonality and leaves the trend. That is the whole content of the thick line.

In the next lecture we take both of them out arithmetically, and that turns out to explain something about today.

THE MATHEMATICS

Stationarity, differencing and autocorrelation

20 minutes · examinable

The question

Every model you have fitted in this course assumes that the data it was trained on and the data it will meet are *the same kind of thing*.

For a time series, what exactly does that assumption say — and does our series satisfy it?

> **WHY YOU NEED THE ANSWER**
>
> The property has a name, our series does not have it, and the repair for not having it is the same operation that will explain why copying last week was so hard to beat.

Stationarity, strictly

A series $\{y_t\}$ is **strictly stationary** if, for every k, every choice of times $t_1 < \cdots < t_k$ and every shift h,

$$(y_{t_1}, \ldots, y_{t_k}) \overset{d}{=} (y_{t_1+h}, \ldots, y_{t_k+h})$$

The whole joint law is invariant under a shift in time. Nothing about the process distinguishes one epoch from another.

This is far more than we need, and far more than any real series has.

Weak stationarity — the version we use

$\{y_t\}$ is **weakly** (or second-order) stationary if three things hold:

$$\mathbb{E}[y_t] = \mu \quad \text{(constant)}$$

$$\mathrm{Var}(y_t) = \sigma^2 < \infty \quad \text{(constant)}$$

$$\mathrm{Cov}(y_t, y_{t+k}) = \gamma_k \quad \text{(depends on } k \text{ only)}$$

Only the first two moments, and only the requirement that the covariance forgets *when* and remembers only *how far apart*.

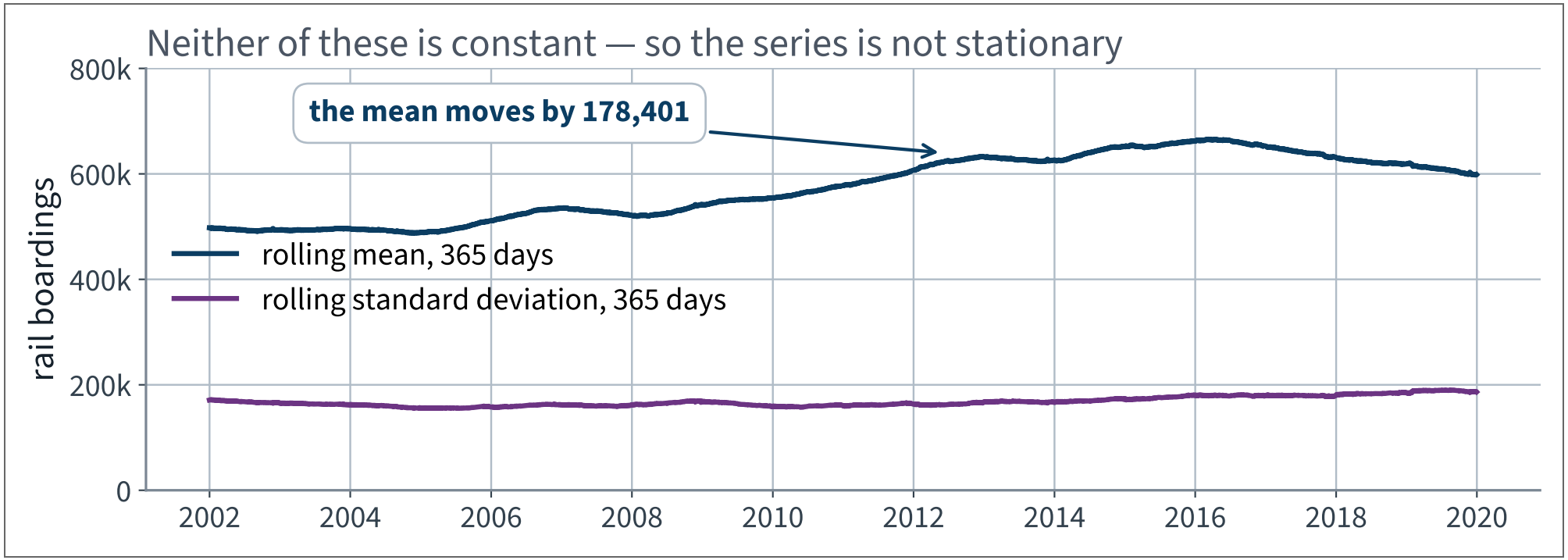
Why a model depends on it

Fitting minimises an average loss over the training rows and we then quote that as an estimate of the loss on future rows. That step is a claim: *the future rows come from the same distribution*.

> For rows indexed by time, “the same distribution” is exactly what weak stationarity asserts about the first two moments. If μ drifts, a model fitted on the past is **systematically** wrong on the future — not noisily wrong, biased.

And it is not a technicality you can ignore: it is the difference between an error that averages out and one that accumulates.

Test the definition directly



✗ The rolling mean moves by more than 170,000 riders across the record. The first condition fails outright.

A statistical test, and what it does not say

The augmented Dickey–Fuller test on the 2016–2019 window returns a statistic of -4.61 with $p \approx 0.0001$, so it *rejects* its null hypothesis.

> **READ WHAT WAS REJECTED**
>
> The null is the presence of a **unit root** — one specific way of being non-stationary. Rejecting it does not establish stationarity; the plot on the previous slide already refuted that.

A test answers the question it was given. Ours was not that question.

The test itself is outside the book and **not examinable**. The habit of asking what a p-value is a p-value *for* is very much examinable.

Differencing

Define the difference operator

$$(\nabla y)_t \;=\; y_t - y_{t-1}$$

and its iterates ∇^d. Applied to a sampled function, ∇ is a discrete derivative: it returns the slope of the series at each step.

One value is lost at the start each time you apply it. Count those losses; the book's `diff` returns `NaN` there and says nothing.

One difference removes a linear trend

Take the series $3, 5, 7, 9, 11$ — exactly linear in t:

$$\nabla: \quad 3,\ 5,\ 7,\ 9,\ 11 \longmapsto 2,\ 2,\ 2,\ 2$$

A linear trend becomes a constant. The constant is the slope.

✓ **And a constant mean is the first condition of weak stationarity, which we have just bought.**

Two differences remove a quadratic one

$$\nabla: \quad 1,\ 4,\ 9,\ 16,\ 25,\ 36 \longmapsto 3,\ 5,\ 7,\ 9,\ 11$$

$$\nabla: \quad 3,\ 5,\ 7,\ 9,\ 11 \longmapsto 2,\ 2,\ 2,\ 2$$

The squares become a linear series, which becomes a constant.

The general statement, and its proof

CLAIM

If p has degree $d \geq 1$ then ∇p has degree exactly $d-1$, so ∇^d annihilates every polynomial trend of degree d.
Monomials suffice, as ∇ is linear.

$$
\begin{aligned}
t^d - (t-1)^d &= \sum_{j=0}^{d-1} \binom{d}{j} (-1)^{d-1-j}\, t^j \\
&= d\, t^{\,d-1} + \text{lower order}
\end{aligned}
$$

Leading coefficient $d \neq 0$, so the degree drops by exactly one. Induct.

This has a name in the book

The number d of differencing rounds is the **order of integration** — the *I* in ARIMA. An ARIMA model differences d times, fits an ARMA model to the result, and adds back what it subtracted when it forecasts.

$$\hat{y}_t \;=\; \sum_{i=1}^{p} \alpha_i \, y_{t-i} \;+\; \sum_{i=1}^{q} \theta_i \, \varepsilon_{t-i}$$

Autoregressive part: a weighted sum of past values — which is precisely the linear model of the previous lecture.
Moving-average part: a weighted sum of past forecast errors.

Seasonal differencing removes a periodic component

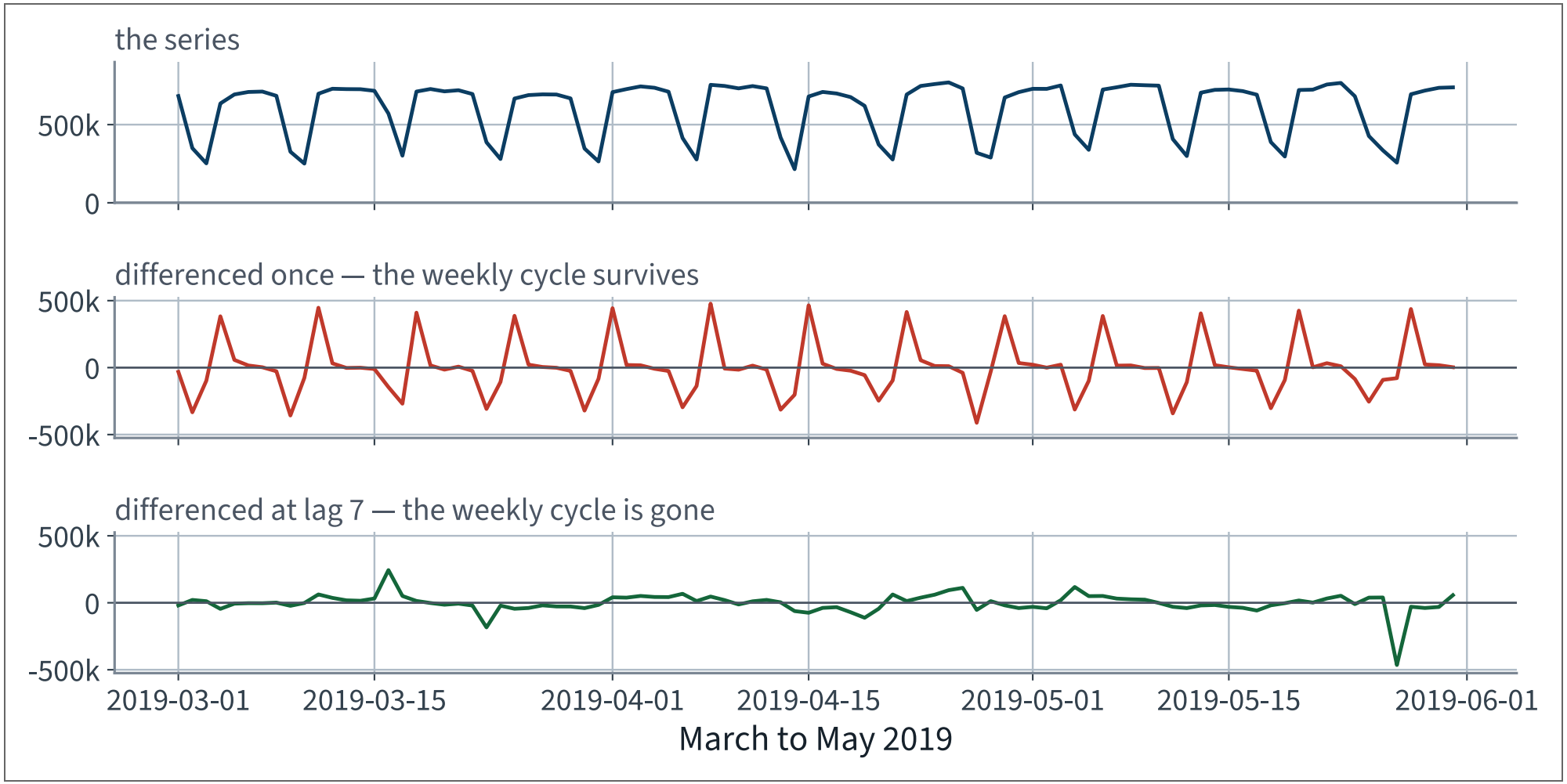
$$(\nabla_s y)_t \;=\; y_t - y_{t-s}$$

Suppose $y_t = c_t + r_t$ where c is periodic with period s, so $c_t = c_{t-s}$ for every t. Then

$$(\nabla_s y)_t \;=\; (c_t - c_{t-s}) + (r_t - r_{t-s}) \;=\; r_t - r_{t-s}$$

✓ The periodic part cancels exactly, with no approximation and no fitted parameter. This is the SARIMA family's contribution.

Both operators, on our own series



Differencing once leaves the weekly cycle intact. Differencing at lag seven removes it.

And differencing can make things *worse*

Series	Standard deviation
the level	183,841
differenced once	**✗ 198,098**
differenced at lag 7	**104,730 ✓**

One difference *increased* the variability by 8%.

That is not a bug and it is not an accident. There is an exact condition, and it is the next slide.

Autocorrelation

For a weakly stationary series, define

$$\rho_k = \frac{\gamma_k}{\gamma_0} = \frac{\mathrm{Cov}(y_t, y_{t+k})}{\mathrm{Var}(y_t)} \in [-1, 1]$$

The correlation of the series with itself, k steps later. $\rho_0 = 1$ by construction, and the whole function $k \mapsto \rho_k$ is the **autocorrelation function**.

Note that it is defined only under stationarity: without it, the covariance depends on t as well as on k and there is no single number to plot.

The variance of a lag-k difference

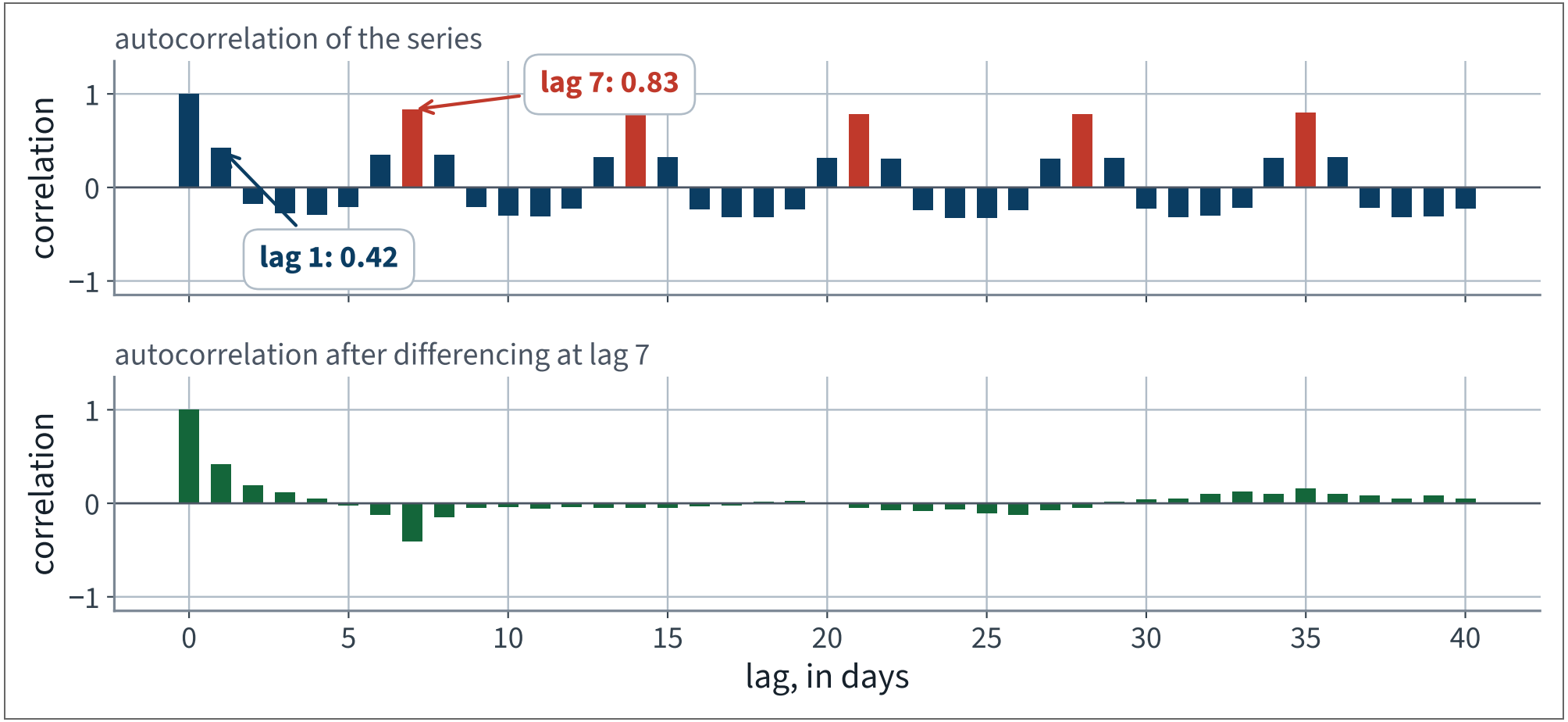
Expand, using stationarity at each step:

$$\begin{aligned}\operatorname{Var}(y_t - y_{t-k}) &= \operatorname{Var}(y_t) + \operatorname{Var}(y_{t-k}) - 2\operatorname{Cov}(y_t, y_{t-k}) \\ &= \sigma^2 + \sigma^2 - 2\sigma^2\rho_k \;=\; 2\sigma^2\,(1-\rho_k)\end{aligned}$$

✓ So differencing at lag k reduces the variance iff $2(1-\rho_k) < 1$:

$$\rho_k \;>\; \tfrac{1}{2}$$

So look at ρ_k before differencing anything



Every multiple of seven stands up. Lag 1 does not reach a half, and that is the whole explanation of the previous slide.

The theory, checked against the data

Lag k	ρ_k	$\sqrt{2\sigma^2(1-\rho_k)}$	Measured
1	0.419	198,236	**198,098 ✓**
7	0.831	106,782	**104,730 ✓**

The lag-1 prediction is right to one part in a thousand; the lag-7 prediction to two per cent. The residual discrepancy is the series failing to be exactly stationary, which we already knew.

✓ $\rho_1 = 0.419 < \frac{1}{2}$, so the first difference must be worse. $\rho_7 = 0.831 > \frac{1}{2}$, so the seventh must be better. Both were, before we knew why.

And now the naive forecast is explained

The forecast “copy the value from seven days ago” has error

$$\hat{y}_t - y_t \;=\; y_{t-7} - y_t \;=\; -(\nabla_7 y)_t$$

Its error *is* the seasonal difference. So its error variance is $2\sigma^2(1-\rho_7)$, and with $\rho_7 = 0.831$ that is $0.34\,\sigma^2$.

> The naive forecast was hard to beat for one reason and one reason only: ρ_7 **is large**. It is not a lucky heuristic. It is the autocorrelation function, read off at a single lag.

One caution before you convert that into an MAE

A Gaussian of standard deviation 104,730 would have mean absolute value $\sqrt{2/\pi}\,\sigma \approx$ **83,562**. Our measured MAE is **55,051**.

> **BECAUSE THE ERRORS ARE NOT GAUSSIAN**
>
> The excess kurtosis of the seven-day difference is **12**: most days are almost exactly like the same day last week, and a handful are catastrophically different. A variance is the wrong summary of that distribution, and so is any conversion that assumes normality.

Which is also the argument for MAE over RMSE, arriving from a second direction.

What the derivation buys you

- Weak stationarity: constant mean, constant finite variance, and a covariance depending only on the lag
- Models assume it, because they quote a training average as an estimate of a future one. Our series does not have it
- ∇^d annihilates a polynomial trend of degree d, because ∇ drops the degree by exactly one
- ∇_s annihilates a component of period s exactly
- $\operatorname{Var}(y_t - y_{t-k}) = 2\sigma^2(1 - \rho_k)$, so differencing helps if and only if $\rho_k > \frac{1}{2}$
- The naive forecast's error is $\nabla_7 y$; its strength is ρ_7

Examinable: state weak stationarity; prove the degree-drop; derive the variance identity and its consequence.

THE METHOD

Baselines, lags, and honest evaluation

40 minutes · examinable

The notebook for this lecture

Open Lecture 15 in Colab

- **Runs on free CPU.** The series is small; nothing here needs an accelerator
- The autocorrelation computed, and the naive forecast's strength predicted from it before it is measured
- The same model evaluated both ways — random split and rolling origin — so the optimism is a number you produce

> **WHAT TO CHANGE**
>
> Shuffle the rows before splitting and re-run the evaluation. The score improves, and nothing warns you. That improvement is the whole argument of the second half of this lecture.

PART TWO

The metric

15 minutes · ending with a number in writing

State the prediction precisely first

$$\hat{y}_{t+1} \;=\; f\left(y_{t-55},\, y_{t-54},\, \ldots,\, y_{t}\right)$$

Given the last eight weeks of daily boardings up to and including today, predict the next day. One number out. Eight weeks is a choice, not a law. We will justify it when we build the dataset.

Three metrics, and what each one buys

$$\mathrm{MAE} = \frac{1}{m}\sum_{i} |\hat{y}_i - y_i| \qquad \mathrm{RMSE} = \sqrt{\frac{1}{m}\sum_{i} (\hat{y}_i - y_i)^2}$$

$$\mathrm{MAPE} = \frac{1}{m}\sum_{i} \frac{|\hat{y}_i - y_i|}{y_i}$$

RMSE punishes a large miss quadratically; MAE does not; MAPE is relative and therefore unit-free.

We choose MAE, and here is the argument

- The units are **passengers**. The operations team can act on “we were out by forty thousand boardings”
- The worst days are holidays, which the calendar already tells us about. We do not want the metric dominated by days we can look up
- RMSE would make one bad holiday worth more than a whole ordinary month

> *Choosing a metric is choosing what to care about. The book states this plainly: if the project suffers quadratically more from large errors, prefer the squared metric. Ours does not.*

We report MAPE beside it, because a percentage is what a stakeholder will repeat back to you.

One trap in MAPE, stated before we use it

MAPE divides by the truth. On a day when ridership collapses, the denominator collapses too, and a modest absolute miss becomes an enormous percentage.

> **WHERE THIS BITES**
>
> Exactly on the holidays, which is exactly where the model is worst. A metric that is most sensitive where the model is weakest will make every improvement look like noise.

We use it as a second opinion, never as the objective.

What would count as good?

Nobody in the room knows yet, and neither do we. So do what the second lecture told us to do: **measure what costs nothing** before committing to anything.

- Predict the same number every day
- Predict today's value again for the next day
- Predict the value from seven days ago

✓ None of these is machine learning. All of them are systems a transit authority could deploy on the first day.

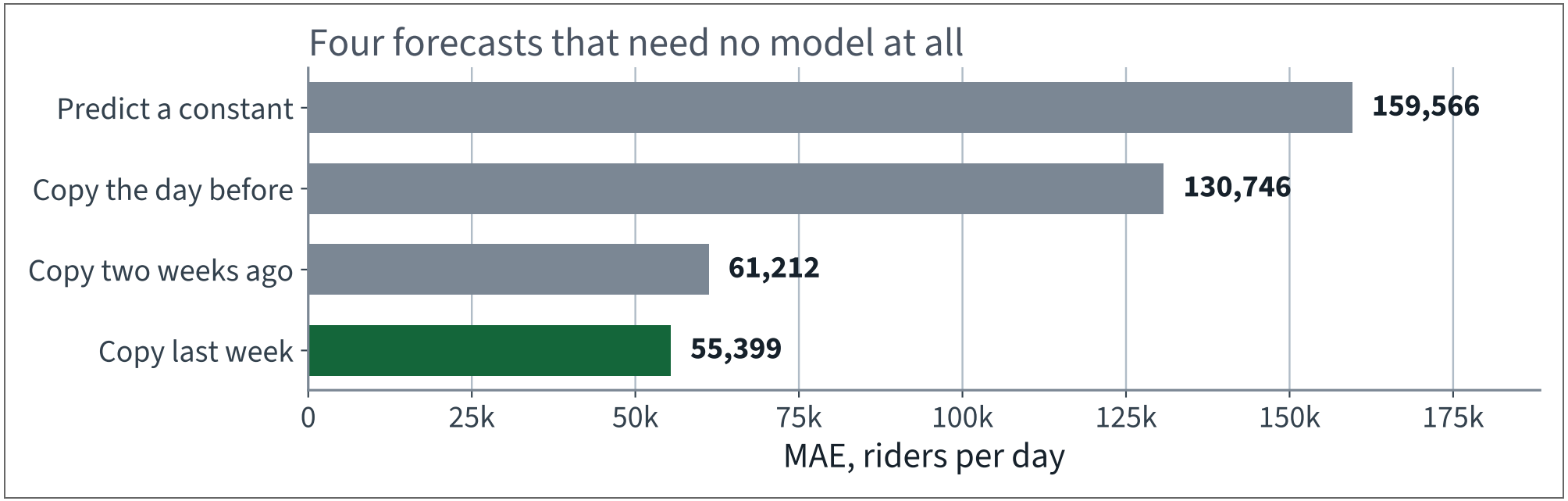
Three lines, three baselines

```
rail = df["rail"]["2016-01":"2019-05"]
target = rail[56:]                        # the days we will be scoring

for name, forecast in [("a constant",   rail[:"2018-12"].mean()),
                       ("copy the day before", rail.shift(1)[56:]),
                       ("copy last week", rail.shift(7)[56:])]:
    print(f"{name:16s} MAE {(target - forecast).abs().mean():>10,.0f}")
```

All three are scored on exactly the rows the models will be scored on, which is the only way the comparison means anything.

What nothing scores



Copying last week is **2.4 times** better than copying the previous day. The seven-day cycle is worth more than recency.

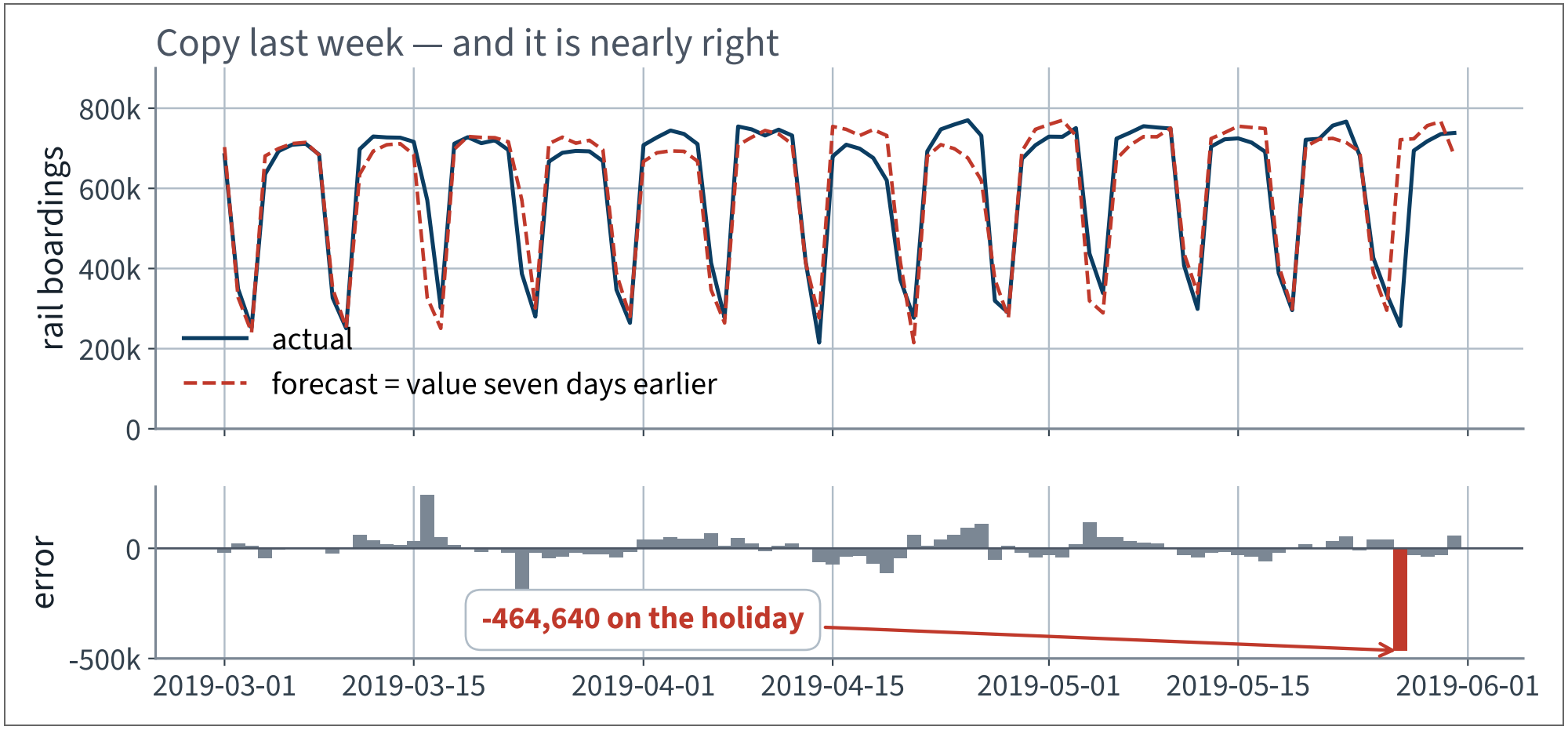
The number to beat

55,399

Mean absolute error of *copy the value from seven days ago*, over 1,191 days.

That is **12.0%** of a typical day. On the three months the book plots it is better still: **42,143** for rail (9.0%) and **43,916** for bus (8.3%).

Copy last week, drawn



✗ One holiday costs 464,640 in a single day — more than eight ordinary days of error put together.

Take this baseline seriously

> **IT IS NOT A STRAW MAN**
>
> It needs no training, no maintenance, no monitoring and no data scientist. It fails only on the days that appear on a public calendar. A transit authority could put it into service in an afternoon.

Any model we build has to be better than this by enough to justify existing. “Better” is not enough; *enough* better is the bar.

✗ Write down what you think *enough* is, before you see any model output. That is the next slide.

The plan

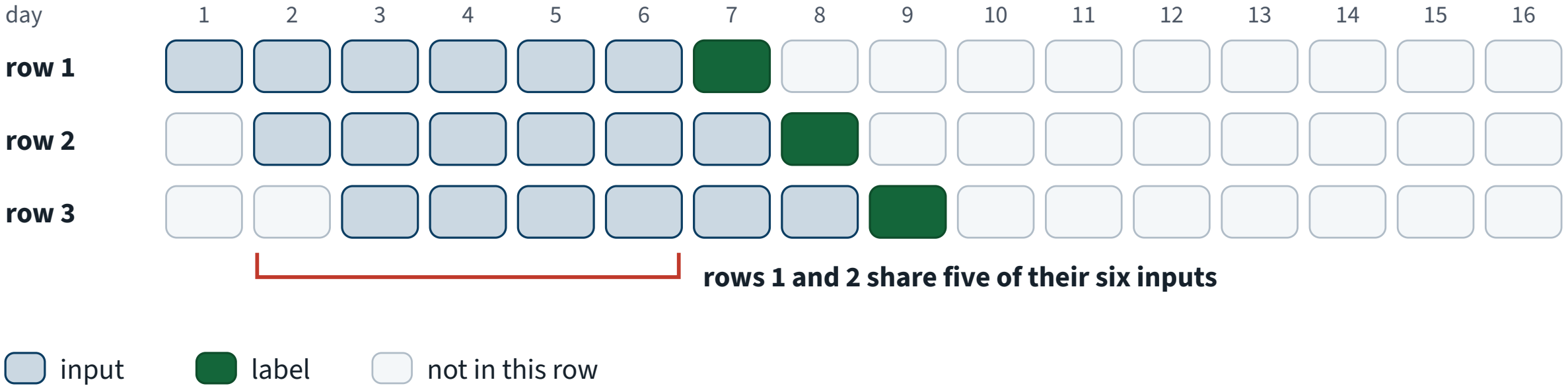
1. Turn one series into a table of rows
2. Check the shapes, twice
3. Fit a linear model on the past 56 days
4. Fit a recurrent network on the same rows
5. Measure, and record the result

Step 1 is the only step that is new. Steps 3 to 5 are the same verbs as every application before this one.

One series becomes a table of overlapping rows

Turning one series into a table of rows

Each row is a window of past days; its label is the day that follows.



drawn with a 6-day window; the lecture uses 56, so consecutive rows share 55 of 56

Inside a row, nothing looks forward. Whether that still holds across rows is decided by the split, not by this table.

Every row is the row above it, shifted by one day. They overlap almost completely.

The dataset class

```
import torch

class TimeSeriesDataset(torch.utils.data.Dataset):
    def __init__(self, series, window_length):
        self.series, self.window_length = series, window_length

    def __len__(self):
        return len(self.series) - self.window_length

    def __getitem__(self, idx):
        end = idx + self.window_length     # first index after the window
        return self.series[idx:end], self.series[end]
```

Nine lines. The subtlety is entirely in `__len__` and in the single index `end`, which is both the end of the window and the position of the label.

Test it on a series whose answer you know

```
>>> toy = torch.tensor([[0], [1], [2], [3], [4], [5]])
>>> for window, target in TimeSeriesDataset(toy, window_length=3):
...     print(window.flatten().tolist(), "->", target.item())
[0, 1, 2] -> 3
[1, 2, 3] -> 4
[2, 3, 4] -> 5
```

✓ Three rows, three targets, none of them inside its own window. Step 4 of the working loop: *test against a case whose answer is known*.

This takes twenty seconds and it is the only thing standing between you and an off-by-one that produces a beautiful score.

Shapes, stated once and asserted

```
window_length = 56
pool = df["rail"]["2016-01":"2019-05"]          # 1,247 days

X, y = windows(pool.values / 1e6, window_length)

assert X.shape == (1191, 56, 1)      # rows, time steps, series
assert y.shape == (1191, 1)
assert len(X) == len(pool) - window_length
```

The recurrent modules want three dimensions: **batch, time, dimensionality**. For one series the last is 1, and forgetting it produces an error message about matrix sizes that names neither time nor batch.

Why the division by a million

Boardings are hundreds of thousands. Dividing by `1e6` puts every value near the interval $[0, 1]$, which is where the default weight initialisation and the default learning rate expect to find their inputs.

> This is scaling, and it is exactly the scaling of the first application — with one difference worth noticing. Dividing by a fixed constant learns **nothing** from the data, so it cannot leak. Compare `StandardScaler`, which learns a mean.

The metric is reported back in passengers by multiplying the error by the same constant.

Why eight weeks

- It is a whole number of seven-day cycles, so every position in the window sees the same phase of the week
- It is long enough to contain a slow change in level
- It is short enough that 1,247 days still yield 1,191 rows

Longer windows cost rows: with a 365-day window this pool gives 882 rows instead of 1,191, and the model has more to learn from less.

It is a hyperparameter. Tune it — but tune it after you can measure it honestly, which is not yet.

One consequence of ρ_k we have not used yet

$\rho_7 = 0.831$ says that any two days a week apart are strongly dependent.

Cross-validation assumes the held-out rows are **independent** of the rows the model was fitted on.

✗ Those two sentences cannot both be acted on.

Read line 3 again

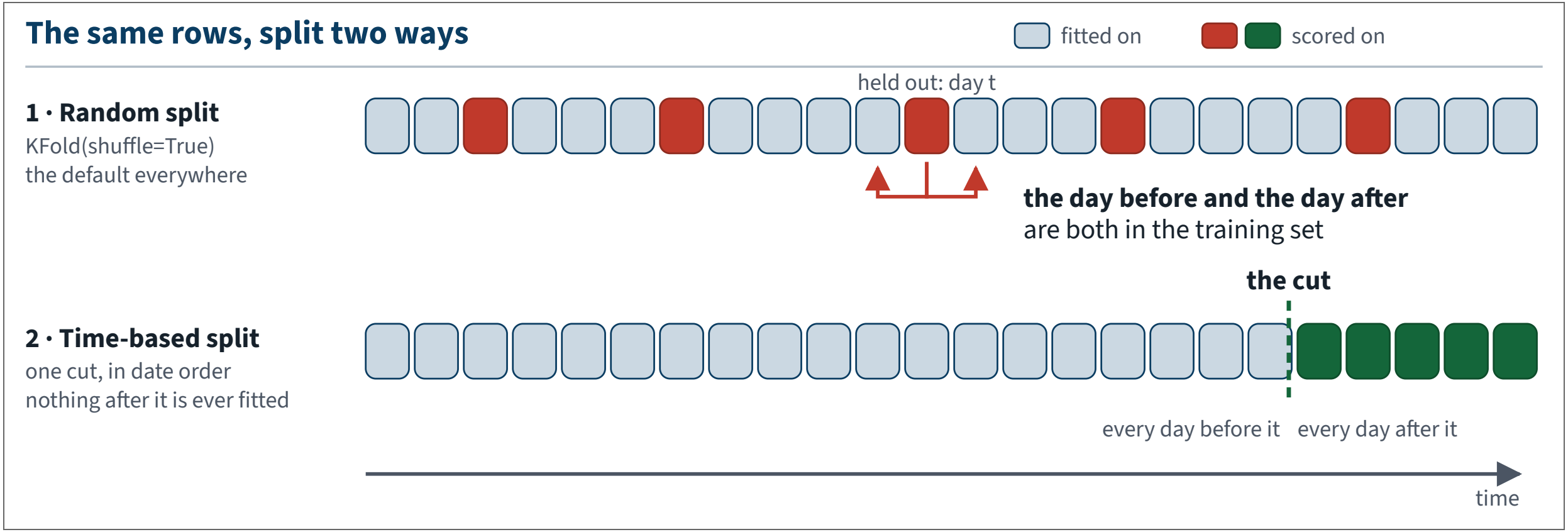
> **FROM LECTURE 2, AND REPEATED IN EVERY DECK SINCE**
>
> “Fixed seeds everywhere; `KFold(shuffle=True, random_state=42)` rather than the default unshuffled split, so two students with differently ordered frames get the same folds.”

That rule was correct. It solved a real problem: a frame sorted differently gave different folds and nobody could see why.

✗ Here it is the bug.

A rule you cannot say the *reason* for is a rule you will apply where it does not hold. This is the clearest example the course contains.

What the two protocols actually do



✗ The upper protocol asks the model to fill a gap. The lower one asks it to forecast. Only one of those is the job.

The argument, stated properly

Cross-validation estimates the risk

$$R(f) \;=\; \mathbb{E}_{(X,Y)}\left[\ell(f_{\mathcal{D}}(X),\, Y)\right], \qquad (X,Y) \perp \mathcal{D}$$

It is unbiased for R under two conditions:

1. the held-out rows are **independent** of the training rows
2. the held-out rows are drawn from the distribution the model will meet **in deployment**

✗ A shuffled split of a time series satisfies neither.

Condition 1 fails, and ρ_k says by how much

$\mathrm{Cov}(y_t, y_{t+k}) = \sigma^2 \rho_k$, which is $0.83\,\sigma^2$ at $k = 7$ and never near zero for $|k| \leq 56$. The rows are dependent by construction.

> Conditioning on neighbours reduces the variance of what remains to be predicted. With $\rho_k > 0$, a model that has been fitted on rows *surrounding* a held-out row faces a smaller conditional variance than one fitted only on rows before it.

Smaller conditional variance means smaller achievable loss, so the estimate is biased **downwards**. Positive autocorrelation fixes the direction of the bias: optimistic in the overwhelming majority of cases.

✓ Not "always". That would be a theorem, and we have not proved one — the argument above is a heuristic about conditional variance, not a bound. What we have is a measurement: on this series, 9–10% after matching training size, test size and the baseline.

Which is precisely why the next three slides control for those alternatives rather than asserting the direction. A lecture whose subject is not asserting things should not assert this one either.

Condition 2 fails, and it is not subtle

In deployment, *every* training day precedes *every* day being forecast.
Under a shuffled split, four training days in five come **after** the day being scored.

> **THE NAME OF THIS LECTURE**
>
> The model is not forecasting. It is being handed the shape of the series around the point it is asked about, and interpolating. That is a different and much easier problem, and nobody is paying for it.

So measure it the other way — one word changed

```python
from sklearn.model_selection import TimeSeriesSplit, cross_val_score

cv = TimeSeriesSplit(n_splits=5)          # was KFold(shuffle=True)
folds = -cross_val_score(model, X, y, cv=cv,
                         scoring="neg_mean_absolute_error")
print(f"MAE {folds.mean():,.0f}   folds {folds.round(0)}")
```

Same model, same rows, same seed, same metric. One argument is different.

Splitter	Linear model, MAE
`KFold(shuffle=True)`	44,925
`TimeSeriesSplit(5)`	**✗ 51,880**

And the split the system will actually face

Fit on everything up to the end of 2018; forecast the five months that follow, one day at a time, never seeing any of them.

Model	Random split	Forecasting forward	Gap
Linear, 56 lags	44,925	**✗ 56,446**	+25.6%
Simple RNN, 32 units	38,224	**✗ 49,073**	+28.4%

+10,848
Riders per day, on the number you wrote on paper.

Now do the thing this course exists for

We have a gap of 28%. **How much of it is the split?**

Three other things changed when we changed protocols, and each of them moves the number on its own:

1. The rows being scored are different rows
2. The training set is smaller, and older
3. The folds have a spread, and we are comparing two means

***✗* Quote 28% without checking those and you have made the same mistake in the other direction.**

Confound 1 · The test rows are harder

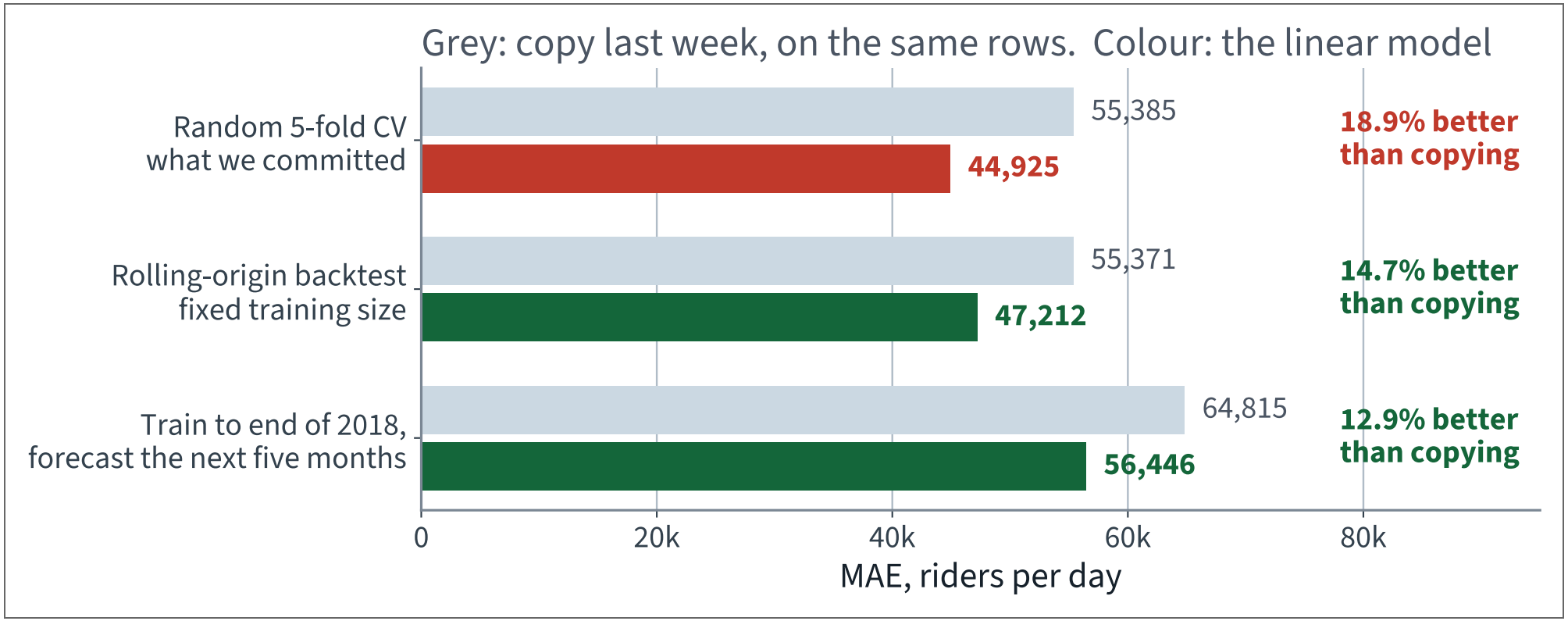
Copying last week needs no fitting, so its error measures only how difficult a set of days is. Measure it on both sets of rows:

Rows scored	Copy last week, MAE
the shuffled folds	55,385
the five months after the cut	**✗ 64,815**

The forecast period is **17% harder** before any model is involved — it contains the winter holidays. A good part of our 28% is that, not leakage.

✓ This is the "we dropped the capped districts" slide from Lecture 2, returning: *any time a number moves, ask first whether the evaluation set moved.*

So put the baseline beside every protocol



The margin over doing nothing falls from **18.9%** to **12.9%** as the protocol becomes honest. That comparison is immune to the test period.

Confound 2 · The training set changed size

`TimeSeriesSplit`'s first fold fits on a fifth of the rows. Its score, 80,506, is not a statement about the protocol; it is a statement about having 238 rows.

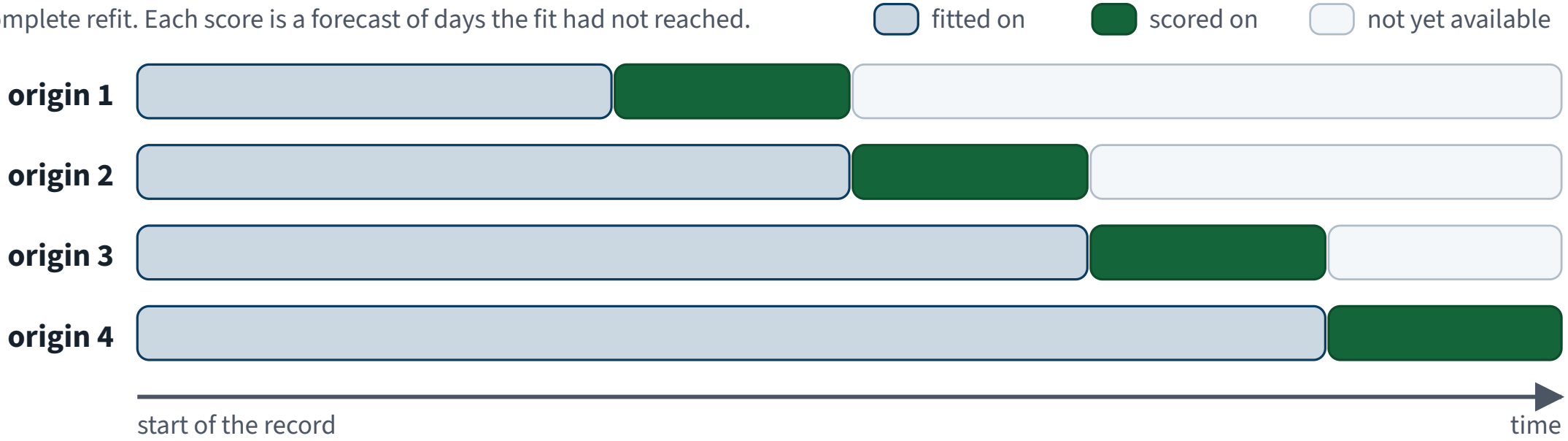
```
TimeSeriesSplit(5) folds:  80506  38448  44838  37991  57615
                           ^^^^^ 238 training rows
```

✓ Repair: a rolling-origin backtest with a fixed training window — 700 rows for every origin, five origins, 98 days scored each time. Now the folds differ only in *when* they are.

Rolling-origin backtesting

Backtesting: re-run the build at four origins

Each row is a complete refit. Each score is a forecast of days the fit had not reached.

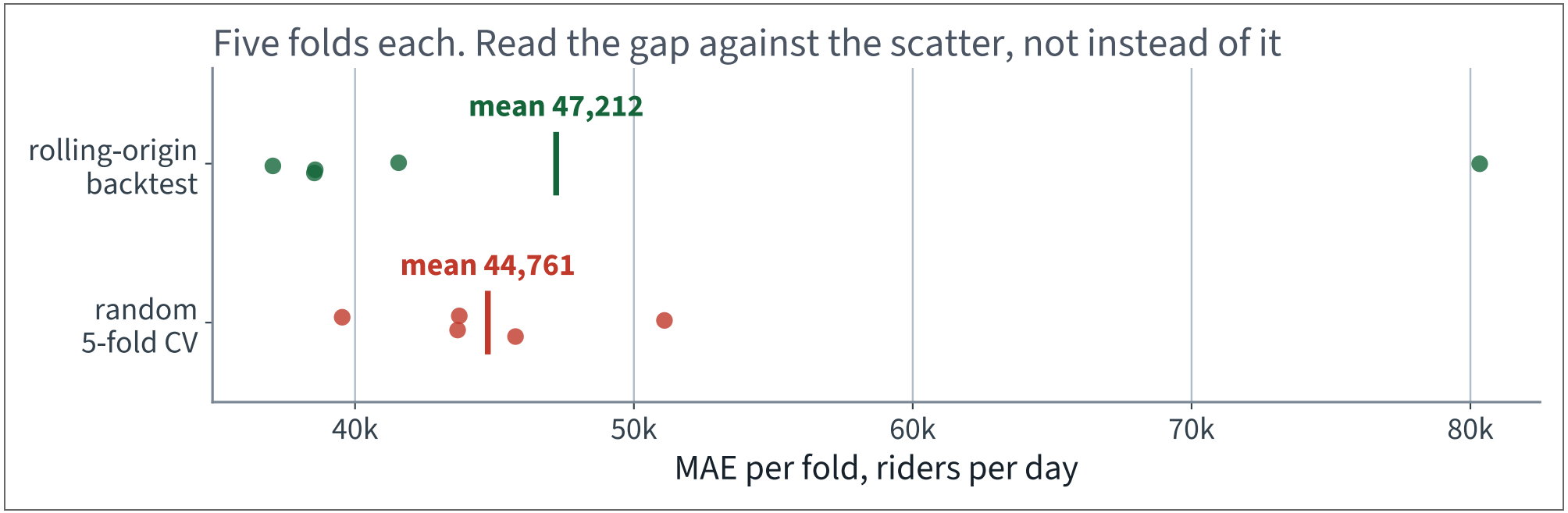


What you get
Four out-of-sample scores instead of one, so you can see the spread as well as the level.

What to watch
Origin 1 is fitted on a quarter of the data. Its score is not comparable.

Four refits, four out-of-sample scores, and a spread you could not get from a single cut.

Confound 3 · The folds have a spread



The random folds span 39,538 to 51,096; the backtest folds span more. A difference in means smaller than that spread is not a difference — the paired-comparison lesson from Lecture 2, unchanged.

The comparison with every confound removed

Same training size (700 rows), same test size (98 rows), same model. Only the *arrangement* differs. Score each against copying last week **on its own rows**, which cancels the difficulty of the period:

Protocol	Model	Naive	Ratio
random draws of 98 rows, 20 of them	48,093	61,461	**✗ 0.782**
rolling origin, 5 origins	47,212	55,371	**0.853 ✓**

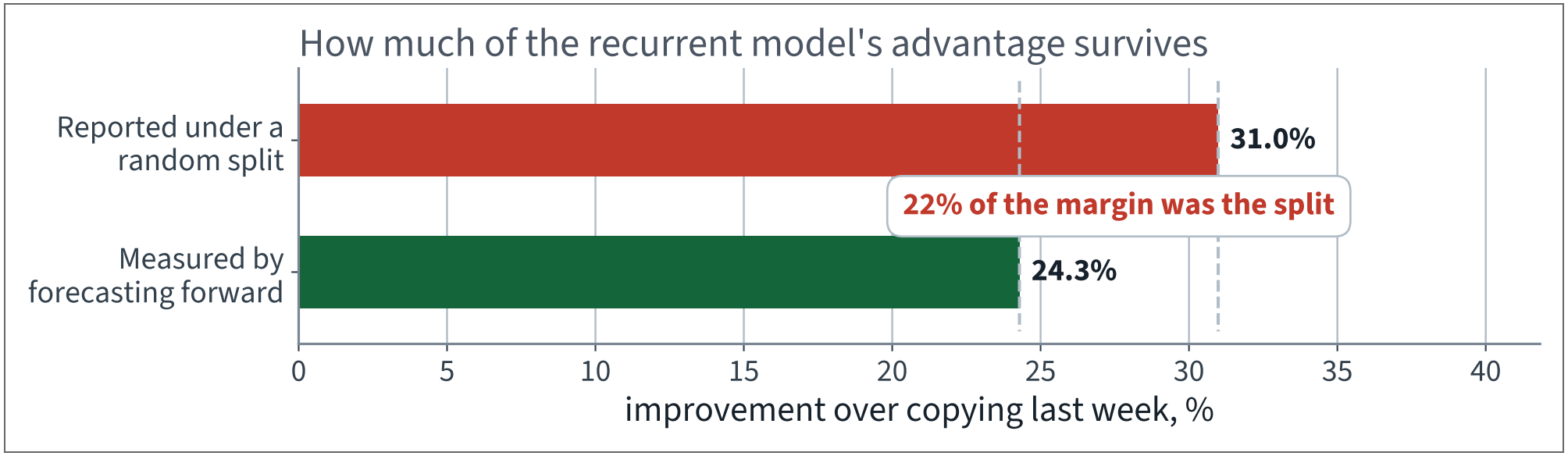
The random split reports the model as **9.0%** more skilful than it is.

The honest verdict, in three numbers

- **28%** — the raw gap on the number you committed. This is what you would have been wrong by, and it is real
- **17%** of that is the forecast period genuinely being harder; the baseline moves too
- **9 to 10%** is the protocol itself, and it survives every control we could apply: matched training size, matched test size, and the baseline computed on the identical rows

Both the recurrent network (9.7%) and the linear model (7.4%) are overstated by roughly the same amount, which is what you would expect if the cause is the split rather than the model.

Put it in the terms the stakeholder will hear



✗ We told them the model was 31% better than doing nothing. It is 24% better. A fifth of the case for building it was an artefact of the evaluation.

Compare with the first leak we measured

	Scaling before the split	Random split on a series
Diagnosed in	Lecture 2	here
Cost, measured	**about one dollar ✓**	**✗ a fifth of the model's value**
Why that size	an invertible affine map, an equivariant estimator, 20,640 rows	ρ_k large at every lag inside the window

✓ Same procedural rule, two orders of magnitude apart in consequence. You cannot tell which case you are in from the number **— only from the structure. That is why the rule is procedural.**

MODEL ONE

Fifty-six weights

10 minutes

The simplest thing that could work

$$\hat{y}_{t+1} = b + \sum_{k=0}^{55} w_k \, y_{t-k}$$

Fifty-six weights and a bias. If the model learns $w_6 = 1$ and every other weight zero, it has reinvented *copy last week*.

✓ That is a useful thing to know before you start: the linear model contains the baseline. It cannot do worse unless we fit it badly.

In code

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import KFold, cross_val_score

Xflat = X.reshape(len(X), -1)          # (1191, 56)

cv = KFold(n_splits=5, shuffle=True, random_state=42)
folds = -cross_val_score(LinearRegression(), Xflat, y.ravel(), cv=cv,
                         scoring="neg_mean_absolute_error")
print(f"MAE {1e6 * folds.mean():,.0f}   per fold {1e6 * folds}")
```

Five folds, shuffled, fixed seed — the standing constraint from the second lecture, applied without modification.

The linear model, measured

44,925

Against **55,399** for copying last week.

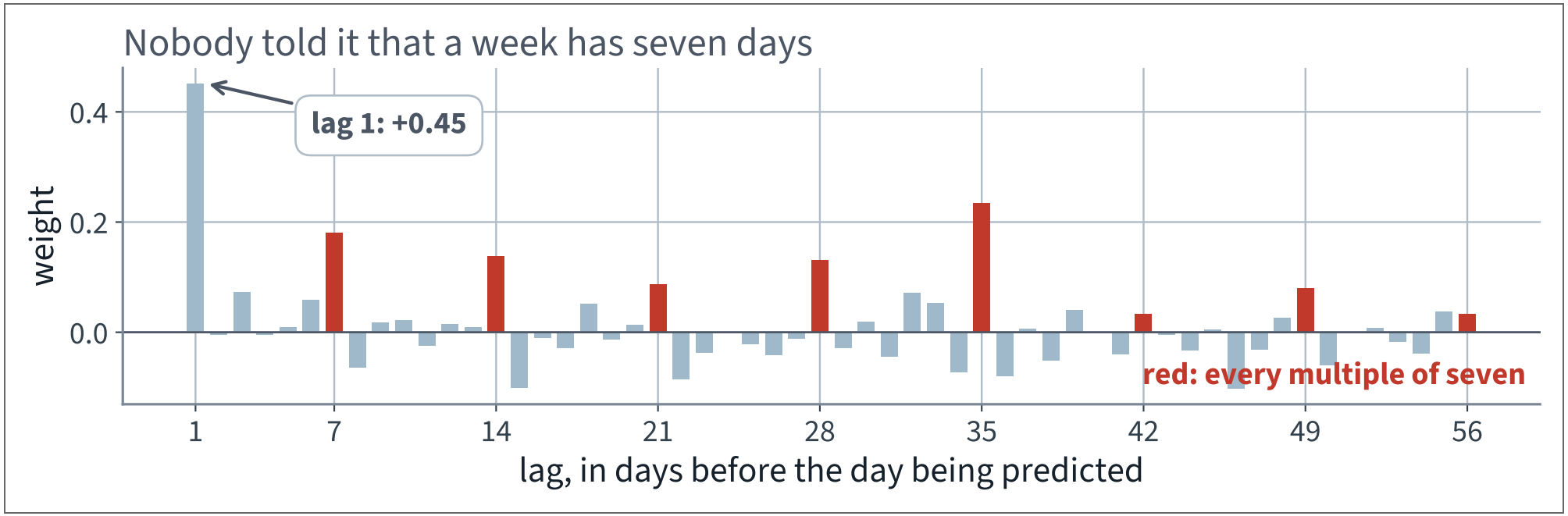
18.9% better, and the five folds run from 39,538 to 51,096. Over twenty different shuffles the mean moves by about 300, so the improvement is far larger than the noise in the estimate.

What did those fifty-six weights learn?

- The single largest weight is at lag 1, at $+0.45$ — the day before still carries the level
- After that, the next four largest are at lags 35, 7, 14 and 28. All seven multiples of seven have a **positive** weight; no other lag does so consistently
- The weights sum to 0.86 and the intercept is about 90,000 riders, so the model shrinks gently towards the mean

✓ It rediscovered the seven-day cycle from the data. Nobody told it that a week has seven days.

Fifty-six weights, and where they went



Every red bar is a multiple of seven, and every red bar is positive. That is the weekly cycle, learned rather than supplied.

From here on, one ground rule

> Every number for the rest of this lecture is a forecast of the five months after 31 December 2018, produced by a model that never saw a single one of those days.

Where a hyperparameter had to be chosen — the optimiser, in our case — it was chosen on the second half of 2018, which is inside the training period. The forecast period was touched once, at the end, per model.
Say what you selected on. Every time.

One refinement, named but not used

Even a clean cut leaks a little: the last training window overlaps the first scored day's history. The repair is to leave a gap — **purging** the overlapping rows and adding an **embargo** of a window length after the cut.

> On this data the effect is small, because the overlap is 56 rows out of 1,040. On a dataset where windows are long relative to the record, it is not.

Purging and embargo are standard practice in financial backtesting and are **outside the book**; not examinable. The idea — that adjacency itself is a channel — is examinable, because we derived it.

Where this sits in the older literature

Model	What it says
AR(p)	today is a weighted sum of the last p days, plus noise
MA(q)	today is a weighted sum of the last q shocks
ARMA(p,q)	both
ARIMA(p,d,q)	both, after differencing d times — which is where today's derivation enters
SARIMA	and seasonal differencing on top

A linear model on lagged features, which is what we fitted, *is* an AR model — fitted by least squares rather than by the Yule–Walker equations, and with whatever extra features you care to add.

The vocabulary is worth recognising; the estimation theory is not examinable here.

BEYOND THE SYLLABUS, FOR CONTEXT

What we did

1. Saw that ordered, autocorrelated rows break the assumption every previous lecture rested on
2. Defined stationarity, strict and weak, and why models depend on it
3. Differenced: once removes a linear trend, d times removes a degree-d polynomial, and seasonal differencing removes a period
4. Computed the autocorrelation, and used it to predict — before measuring — that the naive forecast would be hard to beat
5. Measured the optimism of a random split on autocorrelated data, and replaced it with backtesting on a rolling origin
6. Fitted a linear model on lagged features, which is an AR model by another name

NOT PRESENTED — FOR AFTERWARDS

Exercises

Lecture 15

five, in the style of the written paper

Exercises — Lecture 15

1 For a stationary series, $\mathrm{Var}(X_t - X_{t-h}) = 2\gamma(0)(1 - \rho(h))$. State the condition on $\rho(h)$ under which differencing *increases* the variance. [4]

2 Explain why the seasonal-naive forecast is hard to beat on daily ridership, in terms of the autocorrelation function. [4]

3 State why MAPE is the wrong metric for transit ridership, using a specific day as the example. [4]

Solutions on Lecture 16's deck.

Exercises — Lecture 15 (continued)

4 A cross-validated forecast uses `KFold(shuffle=True)` and reports a good score. Name the two conditions the split violates. [5]

5 In March 2020 the series level falls by three quarters and the model stops working. State whether this is a leak, a bug, or neither, and what it implies for deployment. [4]

Solutions on Lecture 16’s deck.

THE FIVE SET AT THE END OF LECTURE 14

Solutions

Lecture 14

not part of this lecture

Solutions — Lecture 14

1. Two boxes are disjoint and moved further apart. State what IoU and its derivative do, and why no learning…

✓ Both are identically zero; a constant has no descent direction.

- Past contact the intersection is exactly zero for every separation
- The gradient is absent rather than small, so no optimiser can act on it

2. An IoU implementation without a clamp is tested on overlapping boxes only and passes. Give the input that…

✓ Boxes separated on both axes. Assert the result lies in $[0, 1]$ and is zero exactly when the boxes are disjoint.

- Two negative differences multiply to a positive 'intersection'
- A range check alone is necessary and not sufficient — the diagonal case can land inside $[0, 1]$

Solutions — Lecture 14 (2)

3. Average precision is defined with a maximum inside it. State what that maximum repairs, and which earlier…

✓ It replaces the non-monotone precision by its running maximum — the non-monotonicity proved in Lecture 3.

- Precision falls at every false positive and can rise again
- Integrating the raw curve would reward the sawtooth rather than the ranking

4. mAP is a mean over classes of a mean over IoU thresholds. Give one thing each averaging hides.

✓ The class mean hides that a one-instance class weighs as much as a 350-instance one; the threshold mean hides the spread between loose and tight matching.

- Unweighted means ignore support
- A single number in the middle stands for two very different regimes

Solutions — Lecture 14 (3)

5. A team computes AP per image and averages those. State the direction of the error and its cause.

✓ It is optimistic: single images contain few objects and often score 1.0.

- Averaging many easy sub-problems is not the hard problem
- It is the metric averaged per batch rather than over the set

LECTURE 16 · GÉRON CH. 13

Recurrent networks

Recurrent cells, unrolling, and backpropagation through time
Why it is unstable, and what LSTM and GRU gates do about it
Forecasting several steps ahead: recursive, direct and sequence-to-sequence

Run the Lecture 15 notebook first